

SAP ID: 590042185

NAME: Piyush Negi

Experiment - 2: Stages of C Compilation and Execution

1.Objective:

To demonstrate the stages of C program compilation using the -save-temps flag and to write, compile, and execute a C program that calculates the area of a circle.

2. Theory:

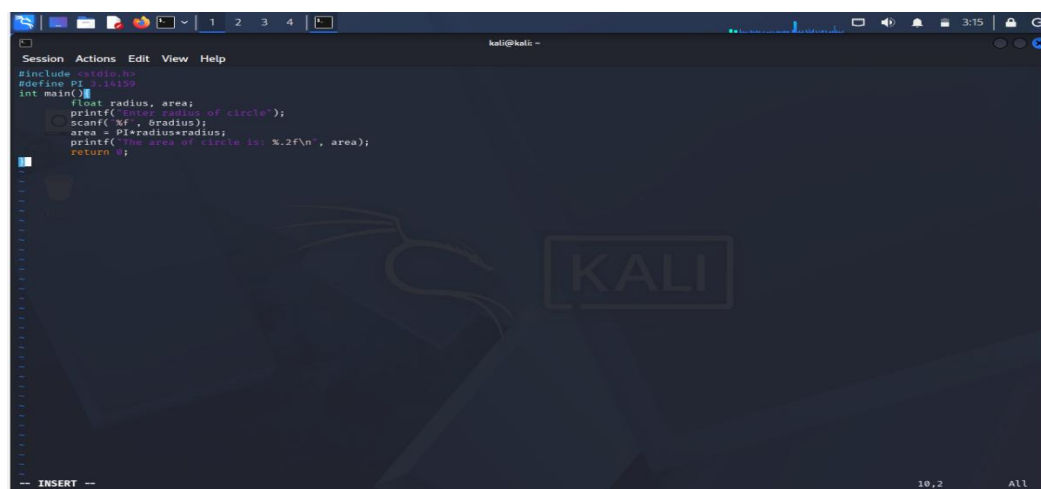
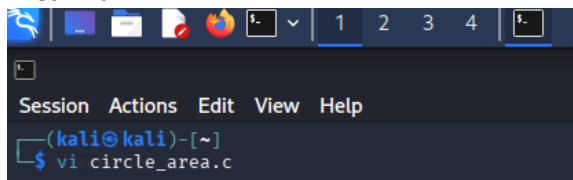
Stages of C Compilation

The process of converting a C source code file into an executable binary involves four primary stages:

- Preprocessing: The preprocessor removes comments, expands macros (like #define), and includes the contents of header files (like stdio.h). It generates an intermediate .i file.
- Compilation: The compiler translates the preprocessed C code into assembly language instructions specific to the target architecture. It generates a .s file.
- Assembly: The assembler converts the assembly language code into machine-level object code (binary). It generates a .o file.
- Linking: The linker combines the generated object code with standard library files to resolve function calls (e.g., printf, scanf) and creates the final executable file.

3. The C Program:

Area of a Circle The following C program calculates the area of a circle using the standard formula
 $\text{Area} = \pi r^2$



4. Demonstrating the Compilation Stages

Compiling with -save-temps:

To observe all the intermediate files generated during the compilation process, the -save-temps option of GCC is used.

Command: gcc -save-temps circle_area.c -o circle_area

- circle_area.c → Source code
- circle_area.i → Preprocessed code
- circle_area.s → Assembly code
- circle_area.o → Object code
- circle_area → Final executable

```
(kali㉿kali)-[~]
$ gcc -save-temps circle_area.c -o circle_area

(kali㉿kali)-[~]
$ ls
circle_area  circle_area.i  circle_area.s  Documents  Music  Projects  Templates
circle_area.c  circle_area.o  Desktop      Downloads  Pictures  Public    Videos

(kali㉿kali)-[~]
$
```

Analysis of Intermediate Files

The preprocessed .i file

circle_area.i (Preprocessed Code):

The .i file contains the preprocessed source code. The file becomes much larger because the contents related to stdio.h are included. Comments are removed and the macro PI is replaced with its value 3.14159.

1. less circle_area.i

```
(kali㉿kali)-[~]
$ less circle_area.i
```

```
Session Actions Edit View Help
extern int fclose (FILE *__stream) __attribute__((__nonnull__(1)));

extern FILE *fopen (const char *__command, const char *__modes)
  __attribute__((__malloc__)) __attribute__((__malloc__(fclose, 1)));

extern char *ctermid (char *s) __attribute__((__nothrow__, __leaf__))
  __attribute__((__access__((write_only, 1))));
# 991 "/usr/include/stdio.h" 3 4
extern void flockfile (FILE *__stream) __attribute__((__nothrow__, __leaf__)) __attribute__((__nonnull__(1)));

extern int ftrylockfile (FILE *__stream) __attribute__((__nothrow__, __leaf__)) __attribute__((__nonnull__(1)));

extern void funlockfile (FILE *__stream) __attribute__((__nothrow__, __leaf__)) __attribute__((__nonnull__(1)));
# 949 "/usr/include/stdio.h" 3 4
extern int _uflow (FILE *);
extern int _overflow (FILE *, int);
# 973 "/usr/include/stdio.h" 3 4
# 2 "circle_area.c" 2
# 3 "circle_area.c"
int main()
{
    float radius, area;
    printf("Enter radius of circle");
    scanf("%f", &radius);
    area = 3.14159*radius*radius;
    printf("The area of circle is: %.2f\n", area);
    return 0;
}
(END)
```

2. head -50 circle_area.i

```
(kali㉿kali)-[~]  
$ head -50 circle_area.i
```

```
kali@kali -  
Session Actions Edit View Help  
# 28 "/usr/include/stdio.h" 3 4  
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4  
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4  
# 1 "/usr/include/features.h" 1 3 4  
# 415 "/usr/include/features.h" 3 4  
# 1 "/usr/include/features-time64.h" 1 3 4  
# 20 "/usr/include/features-time64.h" 3 4  
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4  
# 21 "/usr/include/features-time64.h" 2 3 4  
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4  
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4  
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4  
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4  
# 22 "/usr/include/features-time64.h" 2 3 4  
# 416 "/usr/include/features.h" 2 3 4  
# 523 "/usr/include/features.h" 3 4  
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4  
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4  
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4  
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4  
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4  
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4  
# 524 "/usr/include/features.h" 2 3 4  
# 547 "/usr/include/features.h" 3 4  
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4  
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4  
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4  
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4  
# 548 "/usr/include/features.h" 2 3 4  
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4  
# 29 "/usr/include/stdio.h" 2 3 4  
  
# 1 "/usr/lib/gcc/x86_64-linux-gnu/15/include/stddef.h" 1 3 4  
# 229 "/usr/lib/gcc/x86_64-linux-gnu/15/include/stddef.h" 3 4  
  
# 229 "/usr/lib/gcc/x86_64-linux-gnu/15/include/stddef.h" 3 4  
typedef long unsigned int size_t;  
# 35 "/usr/include/stdio.h" 2 3 4
```

3. grep -n "3.14159" circle_area.i

```
(kali㉿kali)-[~]  
$ grep -n "3.14159" circle_area.i  
841: area = 3.14159*radius*radius;  
  
(kali㉿kali)-[~]  
$
```

The assembly .s file

circle_area.s (Assembly Code):

The .s file contains assembly-language instructions generated by the compiler from the preprocessed C program. These instructions are specific to the target processor architecture.

1. less circle_area.s

```
(kali㉿kali)-[~]  
$ less circle_area.s
```

```
kali@kali: ~  
Session Actions Edit View Help  
leaq .LC0(%rip), %rax  
movq %rax, %rdi  
movl $0, %eax  
call printf@PLT  
leaq -8(%rbp), %rax  
leaq .LC1(%rip), %rdx  
movq %rax, %rsi  
movq %rdx, %rdi  
movl $0, %eax  
call __isoc23_scanf@PLT  
movss -8(%rbp), %xmm0  
pxor %xmm1, %xmm1  
cvtss2sd %xmm0, %xmm1  
movsd .LC2(%rip), %xmm0  
mulsd %xmm0, %xmm1  
movss -8(%rbp), %xmm0  
cvtss2sd %xmm0, %xmm0  
mulsd %xmm1, %xmm0  
cvtss2ss %xmm0, %xmm0  
movss %xmm0, -4(%rbp)  
pxor %xmm2, %xmm2  
cvtss2sd -4(%rbp), %xmm2  
movq %xmm2, %rax  
leaq .LC3(%rip), %rdx  
movq %rax, %xmm0  
movq %rdx, %rdi  
movl $1, %eax  
call printf@PLT  
movl $0, %eax  
leave  
.cfi_def_cfa 7, 8  
ret  
.cfi_endproc  
.LFE0:  
.size main, .-main  
.section .rodata  
.align 8  
.LC2:  
.long -266631570  
.long 1074340345  
.ident "GCC: (Debian 15.2.0-17) 15.2.0"  
.section .note.GNU-stack,"",@progbits  
(END)
```

2.head -50 circle_area.s

```
(kali㉿kali)-[~]
└─$ head -50 circle_area.s
.file "circle_area.c"
.text
.section .rodata
.LC0:
.string "Enter radius of circle"
.LC1:
.string "%f"
.LC3:
.string "The area of circle is: %.2f\n"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
leaq .LC0(%rip), %rax
movq %rax, %rdi
movl $0, %eax
call printf@PLT
leaq -8(%rbp), %rax
leaq .LC1(%rip), %rdx
movq %rax, %rsi
movq %rdx, %rdi
movl $0, %eax
call __isoc23_scanf@PLT
movss -8(%rbp), %xmm0
pxor %xmm1, %xmm1
cvtss2sd %xmm0, %xmm1
movsd .LC2(%rip), %xmm0
mulsd %xmm0, %xmm1
movss -8(%rbp), %xmm0
cvtss2sd %xmm0, %xmm0
mulsd %xmm1, %xmm0
cvtss2sd %xmm0, %xmm0
movss %xmm0, -4(%rbp)
```

The object .o file

circle_area.o (Object Code):

The .o file contains machine-level object code generated by the assembler. It is a binary file and is not normally readable using a standard text editor. It is later passed to the linker to produce the executable program.

1. file circle_area.o

```
(kali㉿kali)-[~]
└─$ file circle_area.o
circle_area.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
```

2. ls -l circle_area.o

```
(kali㉿kali)-[~]  
$ ls -l circle_area.o  
-rw-rw-r-- 1 kali kali 1712 Aug 19 03:21 circle_area.o
```

3. objdump -d circle_area.o

```
(kali㉿kali)-[~]  
$ objdump -d circle_area.o  
  
circle_area.o:      file format elf64-x86-64  
  
Disassembly of section .text:  
  
0000000000000000 <main>:  
  0:  55                push    %rbp  
  1:  48 89 e5          mov     %rsp,%rbp  
  4:  48 83 ec 10       sub     $0x10,%rsp  
  8:  48 8d 05 00 00 00 lea     0x0(%rip),%rax      # f <main+0xf>  
  f:  48 89 c7          mov     %rax,%rdi  
 12:  b8 00 00 00 00    mov     $0x0,%eax  
 17:  e8 00 00 00 00    call   1c <main+0x1c>  
 1c:  48 8d 45 f8       lea     -0x8(%rbp),%rax  
 20:  48 8d 15 00 00 00 lea     0x0(%rip),%rdx      # 27 <main+0x27>  
 27:  48 89 c6          mov     %rax,%rsi  
 2a:  48 89 d7          mov     %rdx,%rdi  
 2d:  b8 00 00 00 00    mov     $0x0,%eax  
 32:  e8 00 00 00 00    call   37 <main+0x37>  
 37:  f3 0f 10 45 f8    movss  -0x8(%rbp),%xmm0  
 3c:  66 0f ef c9       pxor    %xmm1,%xmm1  
 40:  f3 0f 5a c8       cvtss2sd %xmm0,%xmm1  
 44:  f2 0f 10 05 00 00 movsd   0x0(%rip),%xmm0      # 4c <main+0x4c>  
 4b:  00  
 4c:  f2 0f 59 c8       mulsd   %xmm0,%xmm1  
 50:  f3 0f 10 45 f8    movss  -0x8(%rbp),%xmm0  
 55:  f3 0f 5a c0       cvtss2sd %xmm0,%xmm0  
 59:  f2 0f 59 c1       mulsd   %xmm1,%xmm0  
 5d:  f2 0f 5a c0       cvtsd2ss %xmm0,%xmm0  
 61:  f3 0f 11 45 fc    movss   %xmm0,-0x4(%rbp)  
 66:  66 0f ef d2       pxor    %xmm2,%xmm2
```

Final Executable

circle_area (Executable):

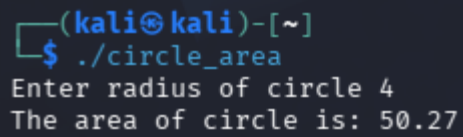
The final executable file is produced after the linking stage. It contains the linked machine code and can be executed by the operating system.

- file circle_area

```
(kali㉿kali)-[~]  
$ file circle_area  
circle_area: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=5c17cd65c1be49e5d9d14eaa50876887006d60ac, for GNU/Linux 3.2.0, not stripped
```

Output:

Command- ./circle_area

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The user enters ./circle_area. The program prompts "Enter radius of circle" and the user enters 4. The program outputs "The area of circle is: 50.27".

```
(kali@kali)-[~]  
$ ./circle_area  
Enter radius of circle 4  
The area of circle is: 50.27
```

Conclusion:

The different stages of C program compilation—preprocessing, compilation, assembly, and linking—were successfully studied using the GCC -save-temps option. The generated .i, .s, and .o intermediate files were examined. Finally, the executable program was run successfully and the area of a circle was calculated correctly.